Torneo de Komikan Concurrente

Informe Tarea 2

Sistemas Operativos

Nicolás Gutiérrez
e-mail: nicolas.gutierrez6@mail.udp.cl

Octubre de 2026

Índice

1. Diseño del esquema de sincronización **2**

2. Análisis de riesgos de concurrencia **3**

3. Evaluación **3**

1. Diseño del esquema de sincronización

Hilos. El proceso tiene un hilo principal, un hilo *vigilante* de SIGINT, un hilo *Matchmaker*, un hilo por jugador y, por cada mesa activa, un hilo *Juez* más 13 hilos de ficha (1 Puma y 12 Perros). Todo el estado compartido se protege con primitivas POSIX (`pthread_mutex`, `pthread_cond`, `sem_t`).

Tablero y exclusión mutua. El tablero sigue la figura del enunciado: 28 nodos (25 casillas 5×5 unidas solo en horizontal y vertical, y 3 nodos del trono), con coordenadas (x, y) donde x es la columna e y la fila ($y = 4$ es la fila junto al trono). Cada nodo tiene su propio `pthread_mutex_t` y una lista de adyacencias y de saltos válidos, construida una sola vez (`pthread_once`) y luego de solo lectura. Un movimiento (`board_move`) bloquea los nodos de origen, destino y, si es captura, el de la casilla saltada (máximo 3), *siempre en orden ascendente de id*. Bajo esos locks se valida todo (la pieza sigue en el origen, el destino está libre, la captura es legal) y se aplica el cambio de forma atómica. Si dos fichas apuntan a la misma casilla en el mismo instante, los locks las serializan y la segunda ve el destino ocupado y falla. Las fichas siguen un patrón *leer–decidir–validar*: obtienen sus opciones con una instantánea tomada bajo los locks del vecindario (`board_moves`) y `board_move` revalida al ejecutar, por lo que una decisión desactualizada nunca corrompe el tablero. Las posiciones de las piezas son `atomic` para poder leerlas sin lock.

Matchmaker y mesas. Los jugadores llegan a una cola enlazada protegida por un mutex; el Matchmaker duerme en una variable de condición hasta que hay al menos 2 jugadores, los empareja (rol Puma/Perros al azar) y hace `sem_wait` sobre un semáforo inicializado en M. Si las M mesas están ocupadas, el Matchmaker queda bloqueado sin consumir CPU. Al terminar, el Juez hace `sem_post`, liberando el cupo. Un contador protegido por mutex registra el pico de mesas activas, que nunca superó M en las pruebas.

Juez y tiempo real. Cada ficha espera su retardo aleatorio (50–300 ms, configurable) con `pthread_cond_timedwait`, de modo que se despierta de inmediato al terminar la partida. Tras cada jugada válida la ficha notifica al Juez por una variable de condición; el Juez evalúa victoria (6 capturas, Puma sin movimientos), abandono o tope de 300 jugadas (empate), y registra la matriz del tablero cada 10 jugadas (o en cada una con `TORNEO_VERBOSE=1`), además de los eventos clave (capturas, inicio, fin). Una jugada solicitada que no cumple las reglas se rechaza y se registra como `MOVE RECHAZADA`, sin abortar la partida.

Resiliencia. Una desconexión activa un flag atómico del jugador; sus fichas lo detectan, avisan al Juez y este declara ganador al rival por abandono. Como el Juez hace `join` de todas las fichas antes de destruir los mutex del tablero, ninguna primitiva queda retenida. Para SIGINT, el *handler* (instalado con `sigaction`) solo hace `write()` a un pipe, que es seguro en señales; el hilo vigilante, bloqueado en `read()`, ejecuta el cierre: marca el servidor como apagado, aborta las mesas activas, cancela a los jugadores, hace `sem_post` para despertar al Matchmaker y luego se hacen `join`, `free` y destrucción de mutex, condvars, semáforo y descriptores del pipe. El programa termina con código 0.

2. Análisis de riesgos de concurrencia

Deadlock. El escenario típico es que dos fichas necesiten dos casillas en orden inverso (A→B y B→A). Se previene imponiendo un orden total sobre los locks de nodos (ascendente por id, también en `board_dump` y `board_check`, que bloquean los 28), lo que elimina la espera circular. Una ficha nunca duerme ni espera otra primitiva mientras sostiene un lock de nodo. Para los mutex de servidor, mesa y jugador se usa un orden fijo (`srv` → `jugador` y `srv` → `mesa`), y el Juez nunca toma el mutex del servidor con el de la mesa tomado.

Livelock. Si las fichas reintentaran sin pausa, podrían chocar indefinidamente. Aquí, tras un fallo, cada ficha espera su siguiente retardo aleatorio e independiente (50–300 ms), lo que rompe la sincronía. Además, una partida donde nadie logra ganar se corta con el tope de 300 jugadas.

Starvation. Los locks de nodo se mantienen solo microsegundos y ninguna ficha duerme sosteniéndolos, de modo que ninguna puede monopolizar una casilla. La cola de jugadores es FIFO, y como el Matchmaker es el único hilo que hace `sem_wait`, no hay competencia por el semáforo.

Otros. Para evitar *lost wakeups* toda espera en condvar usa un predicado en `while` (flag `dirty`, `over`, estado del jugador) protegido por el mismo mutex. En señales no se usan mutex dentro del handler. Los hilos jugador y de ficha terminan siempre y se hace `join` de todos, sin hilos huérfanos.

3. Evaluación

Granularidad. Un mutex por nodo permite que movimientos en zonas distintas del tablero ocurran en paralelo; un mutex único por tablero sería más simple y obviamente correcto, pero serializaría a las 13 fichas. El costo es la complejidad del orden de locks y de bloquear conjuntos de nodos. Dado que cada ficha actúa solo cada 50–300 ms, la contención real es baja y el rendimiento está dominado por los retardos y no por los locks; elegimos este diseño porque es el que mejor evidencia la exclusión mutua fina y escala bien al subir el número de mesas.

Hilos. Se usan $13M$ hilos de ficha más los de jugadores y Jueces. Es costoso para M grande, pero mantiene el modelo fiel al enunciado (cada pieza decide de forma independiente) y simple de razonar.

Validación. Se ejecutó el archivo de casos de prueba entregado con la tarea (12 casos): jugadas normales y de captura, colisiones sobre una misma casilla, jugadas inválidas (el Juez las rechaza y las registra), movimientos en el trono, una cola de 20 jugadores con 2 mesas, desconexión del Puma y de un Perro, un SIGINT simulado con 10 jugadores y una ráfaga de 500 intentos de movimiento desde 8 hilos. Los 12 casos se ejecutaron sin fallos y los invariantes del tablero se mantuvieron en la ráfaga. Con Valgrind (memcheck) no se detectaron fugas de memoria ni errores.

Limitaciones. El archivo de prueba no incluye resultados esperados, por lo que se verificó que cada caso se procese sin fallos y que el Juez rechace lo ilegal, no un resultado concreto. La validación de cada jugada la ejecuta la ficha dentro de `board_move` y el Juez solo detecta victorias; el tope de 300 jugadas es una decisión propia para garantizar término; y las conexiones del trono siguen nuestra lectura de la figura.